MiniCommerce API Reference

Complete reference for the HTTP API exposed by the backend. The local development server listens on `http://localhost:3001`; all endpoints use the `/api` prefix and use JSON request or response bodies where applicable.

Authentication

Login returns a signed JWT valid for 8 hours. Send it on protected requests as `Authorization: Bearer <token>`. Product creation, updates, and deletion require an admin token. Cart and order routes require a customer account. There are no API endpoints to manage categories.

Health

`GET /api/health`

Public service health check.

200 response

```
{ "status": "ok" }
```

Authentication endpoints

`POST /api/auth/login`

Authenticate with an email and password.

Request body

```
{ "email": "admin@minicommerce.dev", "password": "admin123" }
```

200 response

```
{
  "token": "<jwt>",
  "user": { "id": 1, "email": "admin@minicommerce.dev", "name": "Admin", "isAdmin": true }
}
```

Returns `400` if email or password is missing or not a string, and `401` when credentials are invalid.

`POST /api/auth/register`

Public. Creates a customer account (`isAdmin: false`) and signs the customer in immediately. Requires a non-empty name, email containing `@`, and a password of at least 8 characters. Returns `409` if the email is already registered.

Request body

```
{ "name": "Ada Rossi", "email": "ada@example.com", "password": "password123" }
```

Returns the same `{ "token", "user" }` shape as login, with status `201`.

`GET /api/auth/me`

Requires any valid account token. Returns the authenticated user's JWT payload.

200 response

```
{ "user": { "userId": 1, "email": "admin@minicommerce.dev", "isAdmin": true } }
```

Category endpoints

`GET /api/categories`

Public. Lists categories alphabetically. Each object contains `id`, `name`, `description`, `createdAt`, and `productCount`.

`GET /api/categories/:id`

Public. Returns one category with the same fields as the list endpoint. Returns `400` for an invalid ID and `404` if the category does not exist.

`GET /api/categories/:id/products`

Public. Lists a category's products alphabetically. Each product includes its category name. Returns `400` for an invalid ID and `404` if the category does not exist.

Article (product) endpoints

Product objects returned by the list and detail routes include `id`, `name`, `description`, `categoryId`, `category` (category name), `brand`, `price`, `stock`, `imageUrl`, and `createdAt`.

`GET /api/products`

Public. Lists products. All query parameters are optional and can be combined:

Parameter	Description
`search`	Substring match across name, description, and brand.
`category`	Exact category name.
`categoryId`	Category ID; takes precedence over `category` when valid.
`brand`	Exact brand match.
`minPrice`, `maxPrice`	Inclusive numeric price bounds.
`sort`	Sort field: `name`, `price`, `stock`, `category`, or `createdAt`; defaults to `name`.
`order`	`asc` or `desc`; defaults to `asc`.

Example: `GET /api/products?search=mouse&minPrice=10&sort=price&order=desc`

`GET /api/products/categories`

Public. Returns an alphabetically sorted array of category names, for example `["Accessori", "Monitor"]`.

`GET /api/products/:id`

Public. Returns a single product. Returns `400` for an invalid ID and `404` if no product exists.

`POST /api/products`

Admin-only. Creates a product and returns the created product with status `201`.

`PUT /api/products/:id`

Admin-only. Replaces the product fields with the submitted values and returns the updated product. Returns `400` for an invalid ID or body and `404` if no product exists.

Create/update request body

```
{
  "name": "Wireless Mouse",
  "description": "Bluetooth ergonomic mouse",
  "categoryId": 3,
  "brand": "Example",
  "price": 29.99,
  "stock": 12,
  "imageUrl": "https://example.com/mouse.jpg"
}
```

Use `categoryId` (preferred) or an existing category name in `category`. The name is accepted for compatibility; categories are not created automatically. `name` must be non-empty, `price` must be numeric and at least zero, and `stock` must be a non-negative integer (omitted or empty stock defaults to zero). `description` and `brand` default to empty strings; `imageUrl` defaults to `null`.

`DELETE /api/products/:id`

Admin-only. Deletes a product and returns `204 No Content`. Returns `400` for an invalid ID and `404` if no product exists.

Customer cart endpoints

All cart endpoints require a customer bearer token. Admin tokens receive `403`.

`GET /api/cart`

Returns the current customer's cart as `{ "items": [...], "total": 29.99 }`. Each item contains its cart `id`, `quantity`, expanded `product`, and `subtotal`.

`POST /api/cart/items`

Adds a product to the cart. Repeated additions increase its quantity. `quantity` is optional and defaults to `1`.

```
{ "productId": 5, "quantity": 1 }
```

Returns `404` when the product does not exist.

`PATCH /api/cart/items/:productId`

Sets an item's quantity to a positive integer.

```
{ "quantity": 2 }
```

`DELETE /api/cart/items/:productId`

Removes one product from the customer's cart. Returns `204 No Content`.

`DELETE /api/cart`

Empties the customer's cart. Returns `204 No Content`.

Mock order endpoint

`POST /api/orders`

Customer-only. Accepts the current cart and records an order with status `accepted`; returns `201` and empties the cart. This is a demonstration flow: it does not validate or charge a payment method, contact a payment provider, or decrement product stock. Only the cardholder name and last four digits are stored. The full card number, expiry, and security code are not sent to this endpoint or stored.

```
{ "paymentCardholder": "Ada Rossi", "paymentLastFour": "4242" }
```

An empty cart or missing payment fields returns `400`.

Common errors

Validation and invalid-ID errors generally return `400` with `{ "error": "..." }`. Missing records return `404`. Invalid or missing authentication returns `401`; authenticated non-admin users are forbidden from admin-only

routes (`403`). Unexpected server errors return `500` with an `error` field; development responses may also include `detail`.